Schriftlicher Testplan: Registrierung & Account-Erstellung

Testfall / Szenario	**Was testen wir?(Komponente & Input)**	**Warum testen wir es?(Testziel & Motivation)**	**Wie testen wir es? (Testmethode & Vorgehen)**
1. Best case (Erfolgreiche Registrierung)	Die vollständige Erstellung eines Profils (Student oder Start-up) unter Angabe aller erforderlichen Pflichtdaten wie email, firstName, lastName, password und occupation.	Es muss sichergestellt werden, dass vollständig ausgefüllte Datensätze korrekt persistiert und die Benutzerrollen für die spätere UI-Steuerung fehlerfrei zugewiesen werden.	**Integrationstest (Round-Trip-Verfahren, C-R-Ass-D) im Spring-Boot-Kontext:** 1. **C (Create):** Erzeugen des User-Objekts und Speichern in der PostgreSQL-Datenbank via userRepository.save(user). 2. **R (Read):** Erneutes Auslesen aus der DB anhand der auto-generierten ID mittels findById(id) (geschützt durch ein Optional-Objekt). 3. **Ass (Assertion):** Inhaltlicher Datenabgleich über assertEquals() sowie Prüfung auf unterschiedliche Speicheradressen via assertNotSame(), um echten DB-Zugriff zu beweisen. 4. **D (Delete):** Sofortiges Löschen des Test-Users über deleteById(id), um Datenmüll auf dem Server zu vermeiden.
2. Fehlendes Passwort	Den Registrierungsversuch, wenn das Passwort-Feld im übertragenen UserDTO komplett leer oder null gelassen wurde.	Abfangen unvollständiger Konten. Fehlende Kennwörter würden bei späteren Login-Versuchen zu schweren Systemabstürzen wie einer NullPointerException führen.	**Isolierter Unit-Test mit JUnit 5:** Direkte Übergabe eines manipulierten UserDTO-Objekts an die Business-Logik-Komponente RegistrationControl.registerUser(dto). Überprüfung, ob die Methode den Wert isResult() == false zurückgibt und den Fehlergrund exakt auf PASSWORD_MISSING setzt.

3. Passwort zu kurz	Den Registrierungsversuch mit einem Passwort, welches die vorgegebene Mindestlänge (z. B. weniger als 8 Zeichen) unterschreitet.	Gewährleistung eines minimalen Sicherheitsstandards, um die Konten und eingetragenen Startup-Ideen vor unbefugten Zugriffen zu schützen.	**Isolierter Unit-Test mit JUnit 5:** Übergabe eines UserDTO mit einem unsicheren Kurzpasswort (z. B. "123") an die RegistrationControl. Der Test prüft, ob die Logik den Account abweist und eine entsprechende Fehlermeldung im Resultat generiert.
4. Fehlende E-Mail-Adresse	Den Registrierungsversuch, wenn das E-Mail-Adressfeld (address) im UserDTO leer übergeben wird.	Sicherung der Datenqualität der Plattform. Ohne E-Mail-Adresse sind spätere Benachrichtigungen oder Verknüpfungen von Bewerbungen technisch unmöglich.	**Isolierter Unit-Test mit JUnit 5 (TDD-Ansatz):** Übergabe eines UserDTO mit leerer E-Mail an die RegistrationControl. Der Test schlägt so lange fehl, bis der noch offene // ToDo-Bereich in der RegistrationControl.java um die E-Mail-Validierung erweitert wurde und den Versuch blockiert.
5.E-Mail-Duplikat (Doppelter Account)	Den Registrierungsversuch mit einer E-Mail-Adresse, die bereits einem anderen Benutzerkonto in der Datenbank zugewiesen ist.	Erhaltung der absoluten Datenintegrität. Jede E-Mail-Adresse darf nur einmal im System existieren, um eindeutige Zuordnungen zwischen Start-ups und Studierenden zu garantieren.	**Unit- / Integrationstest der Kontrollschicht:** Der Test stellt sicher, dass die RegistrationControl vor dem Speichern die Existenzprüfung (CheckOnDB) aufruft. Das System muss die Registrierung des Duplikats abweisen, das Speichern verhindern und einen eindeutigen Fehlercode zurückgeben.